LEARNING MODULE

18

Monte Carlo Simulation and the Bootstrap

A simulation answers a model's question with an error that the draws measure themselves: how many draws an answer needs, how to need fewer, and what resampling the days keeps and destroys.

VOLUME III · Probability

BUILDS ON LM1 · LM2 · LM5 · LM12 · LM13 · LM14 · LM15 · LM16 · LM17

LEADS TO LM19 · LM20 · LM22 · LM26 · LM30 · LM32 · LM34 · LM36 · LM38 · LM39 · LM45

TIME ≈ 3 h + lab

LEARNING OUTCOMES

Mastery	*After this module you should be able to:*
☐	**1** explain how a seeded generator makes a simulation reproducible, and bump and reprice with common random numbers
☐	**2** draw from the exponential, normal and empirical distributions by the inverse transform, and normal vectors by colouring
☐	**3** calculate a Monte Carlo estimate, its standard error and its 95% Monte Carlo interval, and explain the $1/\sqrt{n}$ rate
☐	**4** compare Monte Carlo integration with Simpson's rule, and explain why simulation wins in many dimensions
☐	**5** apply antithetic variates, control variates and importance sampling, and predict each one's variance reduction
☐	**6** apply the iid bootstrap to a mean and a five-year growth factor, and explain what resampling days keeps and destroys
☐	**7** implement reproducible simulations and bootstraps with NumPy's `Generator` and SciPy, in chunks

LEARNING MODULE OVERVIEW

- Of TR23's 4,619 overlapping five-year windows of 1,231 sessions, 191 ended below their start (4.1%), all from two episodes. A model of iid normal daily log returns answers 5.49% by a formula; a Monte Carlo simulation of it answers 5.3% and an iid bootstrap of the days 5.6%, each ± 0.2 points with 10,000 draws.
- A seeded generator is a deterministic recursion. `default_rng([1, 20261011])` gives 249 draws at or below −2 in 10,000 on every rerun, and another stream gives 214, against 227.5 expected: a rerun reproduces, only a new stream replicates.
- The inverse transform $F_Y^{-1}(U)$ turns uniform draws into any distribution: exponential waits $-\ln(1-U)/\lambda$, normal draws $\Phi^{-1}(U)$ and TAIEX days $R_{(\lceil uT \rceil)}$; colouring gives normal vectors $L\mathbf{Z}$.
- A Monte Carlo mean has standard error $s_Y/\sqrt{n}$, so ten times the accuracy costs a hundred times the draws. The rate explains LM12's colouring gap of 0.0250: for 4,382 draws the root-mean-square gap is 0.0289.
- On LM5's −5% tail, Simpson's rule reaches a relative error of 1.23×10^{-6} with 257 evaluations, where plain Monte Carlo would need about 3.3×10^{16} draws. In N dimensions a grid errs by $O(n^{-4/N})$ and Monte Carlo by $O(n^{-1/2})$.
- Importance sampling divides the variance of that tail estimate by 10,786. Antithetic paths divide the variance of the five-year mean by 6.06 and a control variate by 11.4, while antithetics gain nothing on a tail probability.
- The iid bootstrap draws days from the empirical distribution. Its mean of T days has variance s^2/T, and its growth factor over m days has mean $(1+\bar{R})^m = 2.1580$, both exactly.
- Resampling keeps the one-day law, 7 April 2025 included, and destroys the clustering: the monthly count of days with $|R| \geq 3\%$ has variance 1.846 in the data and 0.598 in resamples.

1 INTRODUCTION

Volume III has built probability models for TAIEX's returns (LM13 to LM17), and many questions about such a model have no formula. A Monte Carlo simulation answers them by drawing from the model many times. The bootstrap draws instead from the data's own empirical distribution. Both answers carry an error that the draws themselves measure.

MOTIVATING CASE **Under water after five years?**

TR23 is the TAIEX total-return index, dividends reinvested: 5,849 daily returns from 3 January 2003 to 8 October 2026. Five years is 1,231 sessions. Of the 4,619 five-year windows that start on a close in the sample, 191 ended below their start: 4.1%. All 191 started between November 2003 and April 2004 or between June and December 2007 (Exhibit 1). So they record two episodes, and the sample holds fewer than five non-overlapping five-year periods.

A model of independent, normal daily log returns fitted to TR23 answers 5.49% by a formula. A Monte Carlo simulation of that model answers 5.3% (Example 3), and an iid bootstrap of the days 5.6% (Example 10), each ± 0.2 points with 10,000 draws. How far can each answer be trusted, and what does each one assume?

Exhibit 1: Five years later

Growth factor over 1,231 sessions (log scale)



Note: TR23's growth factor P_{t+1231}/P_t over 1,231 sessions, at each window's first close, 2 January 2003 → 10 September 2021 (4,619 windows), log scale. Shaded: the under-water starts, 3 November 2003 → 21 April 2004 (116 starts, 111 under water) and 15 June → 7 December 2007 (122, 80). The worst factor is 0.78 (2004-03-09 → 2009-02-24), the best 3.43 (2021-05-17 → 2026-06-09). Descriptive. Source: FinMind `TaiwanStockTotalReturnIndex` (TAIEX), retrieved 18 October 2026. Computations: `data/lm18_numbers.py` and `LM18_lab.ipynb`.

Sections 2 and 3 draw random numbers from any distribution. Section 4 measures simulation error, and Section 5 compares simulation with numerical integration. Section 6 shrinks the error, Section 7 resamples the days, and Section 8 collects the Python.

NOTE **Notation for this module**

The sample is TR23: closes 2 January 2003 → 8 October 2026, $T = 5{,}849$ daily returns R_t from 3 January 2003, log returns $r_t = \ln(1 + R_t)$, 23.7645 years, $A = 246.12$. Moments divide by T: $\bar{R} = 0.062505\%$, $s = 1.2084\%$, $\bar{r} = 0.05517\%$ and $s_r = 1.2107\%$. Five years is $m = \text{round}(5A) = 1{,}231$ sessions. Quantiles are order statistics $R_{(\lceil uT \rceil)}$ (LM14).

Local symbols: Y a simulated quantity with n independent draws Y_i, their mean $\bar{Y}_n$, standard deviation σ_Y in the model and s_Y over the draws (divisor n); $Y = h(\mathbf{Z})$; p the probability an indicator estimates, $\hat{p}$ its estimate; Y_i^- the antithetic partner (not $\tilde{Y}_i$, which recalls LM12's centred $\tilde{X}$), ρ_- the pair's correlation; X a control with known mean μ_X, $\beta = \sigma_{XY}/\sigma_X^2$, $\hat{\beta}$ its estimate; a a shift and $-k$ a threshold (no bets here); λ an exponential rate per session; L a Cholesky factor (no leverage); N a dimension; B the number of resamples (no Brownian motion); $\mathbb{E}_{\hat{F}}$, $\text{Var}_{\hat{F}}$, $\Pr_{\hat{F}}$ for days drawn from $\hat{F}$; $\bar{R}_{\text{boot}}$ and G_{boot} a resample's mean of T days and growth factor over m days (no star, which marks an optimum); $\hat{\Sigma}_{\text{sim}}$ a simulated covariance matrix, $\|M\|_F$ the Frobenius norm. Indices: i draws, t days, j ranks.

NOTE **Research hygiene**

Every model of TAIEX here is fitted to all of TR23, so its answers are **in-sample**, not forecasts. The windows and monthly counts are **descriptive**. A 95% Monte Carlo interval measures simulation error only, never model error. Resampling ignores the order of days, so a bootstrap is no test on later data (LM22 resamples blocks; LM34 tests on later data).

2 PSEUDO-RANDOM NUMBERS AND SEEDS

- [ ] explain how a seeded generator makes a simulation reproducible, and bump and reprice with common random numbers

A computer draws random numbers by arithmetic.

EXAMPLE 1

Same seed, same answer

Count the draws at or below −2 among 10,000 standard normal draws from `default_rng([1, 20261011])`. Run it twice, then once more with `default_rng([16, 20261011])`.

SOLUTION

The first run gives 249 hits, and the rerun gives 249 again, draw for draw. The second stream gives 214. Since $\Phi(-2) = 0.02275$, the model expects 227.5 hits, and a binomial count has standard deviation $\sqrt{n\,p(1-p)} = 14.9$ by (15.2). Both counts lie within 1.5 standard deviations of 227.5. A rerun reproduces a number; only a new stream replicates the experiment.

The generator

A **pseudo-random number generator** is a deterministic recursion. A **seed** sets its internal state, and each step updates the state and outputs a number. NumPy's `default_rng(seed)` builds a `Generator` on the PCG64 algorithm, whose sequence repeats only after 2^{128} steps (O'Neill 2014; stated without proof).

The `Generator` turns the outputs into uniform draws on $[0, 1)$. A random variable U is **uniform**, $U \sim \text{Unif}(0, 1)$, when $\Pr(U \le u) = u$ for $0 \le u \le 1$. The draws are designed to behave like iid uniform draws, independent as in (16.1). This book does not test generators.

One stream per experiment

Every simulated experiment in this module creates its generator once, at its start, as `default_rng([e, 20261011])`, with `e` the experiment's number. Example 4 alone keeps LM12's seed, because it re-runs that lab. Each experiment has its own generator, so resizing one leaves the others unchanged. Through NumPy's `SeedSequence`, different values of `e` give streams that are independent with very high probability. A generator reseeded inside a loop would repeat its draws.

Rerunning with the same seed **reproduces** a result exactly, which checks the code, not the estimate. Running with another seed **replicates** the experiment, and the spread of the replications shows the simulation error. NumPy promises identical streams only for the same version and build, so the printed numbers rest on the NumPy that `uv.lock` pins.

PITFALL **Bump and reprice with common random numbers**

Bumping and repricing a Monte Carlo value with LM2's central difference (2.11) divides the difference of two simulations by a small step, which magnifies their errors. With **common random numbers**, both runs use the same draws, so most of their errors cancel. For the slope of $\Pr(Z \le -k)$ at $k = 2$ with $\Delta k = 0.01$ and 100,000 draws a side, the standard error falls from 0.0333 to 0.0052 (Problem 11).

導讀 **隨機種子：重現不等於重複實驗**

電腦產生的隨機數（random number）其實是擬隨機數（pseudo-random number）：擬隨機數產生器（pseudo-random number generator）是確定的遞迴（recursion），隨機種子（random seed）決定起始狀態，之後每一步都算得出來。它的輸出設計成像獨立同分布（iid）的均勻分布（uniform distribution）抽樣，這正是蒙地卡羅法（Monte Carlo method）需要的模型。同一個種子重跑一次，每個數字都一模一樣，只證明程式沒變，不是第二次檢驗；換一個種子才是重複實驗，結果之間的差距就是模擬誤差。本模組每個實驗（重跑 LM12 的 Example 4 除外）在開頭建立一次自己的產生器 `default_rng([e, 20261011])`，之後不再重設。用中央差分（central difference）估計導數（derivative）時，兩邊的計算要共用同一組隨機數（common random numbers），誤差才會大部分抵銷。

KNOWLEDGE CHECK 1

A colleague reruns your simulation with the same seed and gets the same estimate. Is that a second check of the estimate?

ANSWER

No. The same seed repeats every draw, so the rerun checks the code, not the simulation error. A new stream replicates the experiment, and the gap between 249 and 214 hits in Example 1 shows the error.

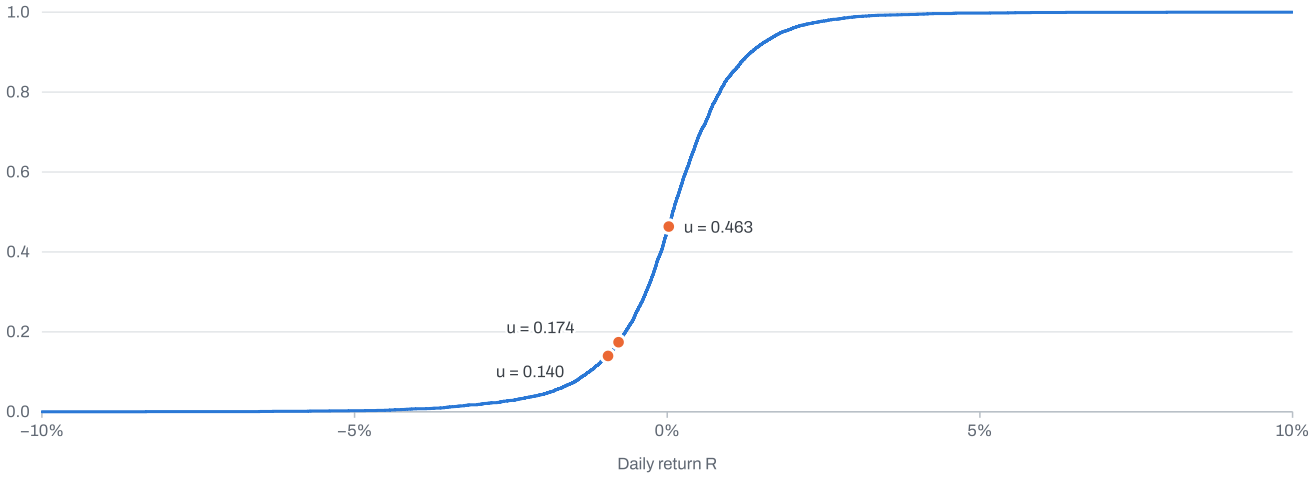
3 FROM UNIFORM DRAWS TO ANY DISTRIBUTION

- ☐ draw from the exponential, normal and empirical distributions by the inverse transform, and normal vectors by colouring

A uniform draw u can be read as a probability level. In Exhibit 2, mark u on the vertical axis of TR23's empirical distribution function $\hat{F}$, (13.12), move right to the first step that reaches u, and read the return below. That return is the empirical quantile $\hat{F}^{-1}(u) = R_{(\lceil uT \rceil)}$ of LM14.

Exhibit 2: Inverse transform on TR23

Empirical distribution function, share of days with return at or below R



Note: the empirical distribution function $\hat{F}$ of TR23's 5,849 daily returns, a step of $1/5{,}849$ at each return. The three points are Example 2's uniforms u placed at the returns they draw, $R_{(818)}$, $R_{(2712)}$ and $R_{(1019)}$. Descriptive. Source: FinMind `TaiwanStockTotalReturnIndex` (TAIEX), retrieved 18 October 2026.

THEOREM **Inverse transform**

Let $U \sim \text{Unif}(0,1)$, and let F_Y be a distribution function with LM14's quantile function $F_Y^{-1}(u) = \min\{y : F_Y(y) \geq u\}$ for $0 < u < 1$. Then

$$\Pr\left(F_Y^{-1}(U) \leq y\right) = F_Y(y) \quad \text{for every } y, \tag{18.1}$$

so $F_Y^{-1}(U)$ has distribution function F_Y.

When F_Y is continuous and strictly increasing, F_Y^{-1} is its ordinary inverse and is increasing. So $F_Y^{-1}(U) \leq y$ exactly when $U \leq F_Y(y)$, an event of probability $F_Y(y)$ (Grinstead and Snell 1997, §5.2, Corollary 5.2). Deep Dive (a) proves the general case (Glasserman 2003, §2.2.1).

DEEP DIVE **(a) Proof of (18.1) for any distribution function** optional · click to expand

Here F_Y is any distribution function: non-decreasing, right-continuous, with limits 0 and 1 (LM13). Fix u in $(0,1)$. The set $\{y : F_Y(y) \geq u\}$ is not empty, because $F_Y \to 1$, and bounded below, because $F_Y \to 0$. Right-continuity carries $F_Y \geq u$ from the set to its infimum, so the infimum is a minimum and $F_Y\left(F_Y^{-1}(u)\right) \geq u$.

Now $F_Y^{-1}(u) \leq y$ if and only if $u \leq F_Y(y)$. If $u \leq F_Y(y)$, then y lies in the set, so the minimum is at most y. If $F_Y^{-1}(u) \leq y$, then $F_Y(y) \geq F_Y\left(F_Y^{-1}(u)\right) \geq u$, since F_Y is non-decreasing. Hence $\Pr\left(F_Y^{-1}(U) \leq y\right) = \Pr\left(U \leq F_Y(y)\right) = F_Y(y)$. ■

The exponential distribution. A waiting time $Y \geq 0$ with $F_Y(y) = 1 - e^{-\lambda y}$ follows $\text{Exp}(\lambda)$, with rate $\lambda > 0$. Solving $u = 1 - e^{-\lambda y}$ gives $F_Y^{-1}(u) = -\ln(1-u)/\lambda$. Its mean is $1/\lambda$ (integrate by parts, LM5), and

its median is $\ln 2/\lambda$, the y with $F_Y(y) = 1/2$.

Normal draws. For the standard normal, $\Phi^{-1}(U) \sim \mathcal{N}(0,1)$, and $\mu + \sigma\Phi^{-1}(U) \sim \mathcal{N}(\mu, \sigma^2)$ by (15.6).

Days from $\hat{F}$. The rank $\lceil uT \rceil$ equals j exactly when $(j-1)/T < u \le j/T$, an interval of length $1/T$. So the inverse transform on $\hat{F}$ draws each of the T days with probability $1/T$: it picks a day uniformly at random.

EXAMPLE 2

A waiting time and a TAIEX day from three uniforms

Experiment 2's generator `default_rng([2, 20261011])` gives `random(3)` = 0.1396965, 0.4634985 and 0.1740834 (seven decimals). Turn them into waiting times between days with $|R| \ge 3\%$ under an exponential model, and into TAIEX days drawn from $\hat{F}$.

SOLUTION

TR23 has 175 days with $|R| \ge 3\%$ (LM14), so the model's rate is $\lambda = 175/5{,}849 = 0.02992$ per session. The waits $-\ln(1-u)/\lambda$ are 5.03, 20.81 and 6.39 sessions. The model's mean wait is $1/\lambda = 33.42$ sessions and its median $\ln 2/\lambda = 23.17$.

For $\hat{F}$, the ranks $\lceil 5{,}849u \rceil$ are 818, 2,712 and 1,019; seven decimals matter, since $5{,}849 \times 0.4634985 = 2{,}711.003$. The draws are $R_{(818)} = -0.9468\%$ (2021-12-14), $R_{(2712)} = +0.0247\%$ (2007-02-01) and $R_{(1019)} = -0.7785\%$ (2005-09-23), the points of Exhibit 2. The waits are model values, while the three returns are real days.

Normal vectors. For N independent standard normal draws $\mathbf{Z}$ and $LL^\top = \Sigma$, (16.7) gives $L\mathbf{Z} \sim \mathcal{N}_N(\mathbf{0}, \Sigma)$, the colouring of LM12 (Glasserman 2003, §2.3.3). Section 4 measures how close a coloured sample comes to Σ.

PITFALL **A uniform draw can be 0**

`Generator.random` returns numbers in $[0, 1)$, and 0 is possible. Then $-\ln U$ is infinite and $\lceil TU \rceil = 0$ is no rank, while $1 - U$ lies in $(0, 1]$. Draw waits as $-\ln(1-U)/\lambda$, and draw ranks with `integers(0, T)`, which returns the indices 0 to $T-1$ directly.

導讀 **逆變換法：把機率水準換成數值**

逆變換法（inverse transform）把均勻分布（uniform distribution）的抽樣 u 當成機率水準，再用分位數函數（quantile function）換成任何分布的抽樣：$F_Y^{-1}(U)$ 的累積分布函數（distribution function）就是 F_Y。指數分布（exponential distribution）的等待時間（waiting time）是 $-\ln(1-U)/\lambda$，常態分布（normal distribution）的抽樣是 $\Phi^{-1}(U)$，多變量常態分布（multivariate normal distribution）的向量則用 L 著色（colouring）。對經驗分布函數（empirical distribution function）$\hat{F}$ 做逆變換，等於從 5,849 個交易日中等機率挑一天，這是第 7 節自助重抽法（bootstrap）的基礎。`random` 可能剛好抽到 0，所以等待時間要用 $-\ln(1-U)/\lambda$，排名則直接用 `integers` 抽。

KNOWLEDGE CHECK 2

Why is $-\ln U/\lambda$ also an $\mathrm{Exp}(\lambda)$ draw, and which return does $u = 0.5$ draw from $\hat{F}$?

ANSWER

If U is uniform, so is $1-U$, so $-\ln U/\lambda$ has the law of $-\ln(1-U)/\lambda$; only $U=0$ must be avoided. For $u = 0.5$ the rank is $\lceil 2{,}924.5 \rceil = 2{,}925$, and $R_{(2925)} = +0.0879\%$ is TR23's median (LM14).

4 THE MONTE CARLO ESTIMATOR AND ITS ERROR

- ☐ calculate a Monte Carlo estimate, its standard error and its 95% Monte Carlo interval, and explain the $1/\sqrt{n}$ rate

Every answer the case asks for is an expectation: a probability is $\mathbb{E}[\mathbf{1}_{\mathcal{A}}]$ by (14.1), and the mean growth factor is $\mathbb{E}G$. For a standard normal Z, $\mathbb{E}h(Z) = \int h(z)\varphi(z)\,dz$ by (5.9), and a path of m days is an integral in m dimensions. The **Monte Carlo estimator** replaces the integral by the average of n independent draws.

THEOREM **The Monte Carlo estimator**

Let $Y_1, \dots, Y_n$ be iid draws of Y with $\mathbb{E}|Y| < \infty$, and $\bar{Y}_n = \frac{1}{n}\sum_i Y_i$. Then $\mathbb{E}\bar{Y}_n = \mathbb{E}Y$, and $\bar{Y}_n \to \mathbb{E}Y$ with probability one as $n \to \infty$. If also $\sigma_Y^2 < \infty$,

$$\operatorname{Var}\bar{Y}_n = \frac{\sigma_Y^2}{n}, \qquad \text{standard error } \frac{s_Y}{\sqrt{n}}. \tag{18.2}$$

The mean and the variance are (17.1), and the convergence is the strong law of LM17 (Durrett 2019, Theorem 2.4.1). The **standard error** $s_Y/\sqrt{n}$ estimates the standard deviation of $\bar{Y}_n$ from the draws themselves. It shrinks like $1/\sqrt{n}$, so ten times the accuracy costs a hundred times the draws.

By the central limit theorem (17.5), $\bar{Y}_n$ is close to normal for large n, so it lies within $1.96\,\sigma_Y/\sqrt{n}$ of $\mathbb{E}Y$ with probability close to 0.95. With s_Y for σ_Y, this gives the **95% Monte Carlo interval**

$$\bar{Y}_n \pm 1.96\,\frac{s_Y}{\sqrt{n}}, \qquad 0 < \sigma_Y^2 < \infty,\ n \text{ large.} \tag{18.3}$$

The interval (18.3) misses $\mathbb{E}Y$ in about one run in twenty. It is approximate: close with thousands of draws, but not when the draws contain few hits of a rare event (Section 5). LM20 develops standard errors and confidence intervals for data. A printed estimate keeps only the digits its standard error supports: 0.0530 ± 0.0022 prints as 5.3% ± 0.2 points.

Other standard errors. An indicator $Y = \mathbf{1}_{\mathcal{A}}$ has $Y^2 = Y$, so $\sigma_Y^2 = p(1-p)$ and the standard error is $\sqrt{\hat{p}(1-\hat{p})/n}$. For near-normal draws, s_Y^2 has relative standard error about $\sqrt{2/n}$ by $\mathrm{Var}(Z^2) = 2$, (14.6). A square root halves a small relative error (LM2's tangent line), so s_Y has standard error about $s_Y/\sqrt{2n}$.

For a quantile or a slope, compute the statistic on 20 equal batches of the draws. For large n its variance falls about like $1/n$, as (17.1) gives for a mean (stated without proof; Glasserman 2003, §9.1.2, for a quantile; LM23 for a slope). So a batch 20 times smaller gives about 20 times the variance, and the standard deviation of the 20 values (divisor 20) over $\sqrt{20}$ approximates the standard error.

EXAMPLE 3

The case by simulation

In the case's model, daily log returns are iid $\mathcal{N}(g, \sigma_r^2)$ with $g = \bar{r} = 0.05517\%$ and $\sigma_r = s_r = 1.2107\%$. Find $\Pr(G < 1)$ for $m = 1{,}231$ sessions exactly, then from 10,000 paths simulated with `default_rng([3, 20261011])`.

SOLUTION

By (1.12), $\ln G = \sum_t r_t$ is a sum of m iid normal log returns, so $\ln G \sim \mathcal{N}(mg, m\sigma_r^2)$ by (16.7), and (17.6) holds exactly. Hence $\Pr(G < 1) = \Phi(-g\sqrt{m}/\sigma_r) = \Phi(-1.5988) = 5.49\%$.

Experiment 3 draws `standard_normal((500, m))` twenty times and sums $g + \sigma_r Z_t$ along each row. Of the 10,000 paths, 530 end under water, so $\hat{p} = 0.0530$ with standard error $\sqrt{0.0530 \times 0.9470/10{,}000} = 0.0022$. The 95% Monte Carlo interval, about 4.9% to 5.7%, covers 5.49%, and the simulation reports 5.3% ± 0.2 points.

Why LM12's colouring gap was 0.0250

LM12's lab coloured 4,382 standard normal vectors with S5's Cholesky factor. Their covariance matrix missed Σ by a relative 0.0250, and the lab left the rate to this module. Let $\mathbf{X}_1, \ldots, \mathbf{X}_n$ be iid $\mathcal{N}_N(\mathbf{0}, \Sigma)$, and let $\hat{\Sigma}_{\text{sim}} = \frac{1}{n}\sum_i \mathbf{X}_i\mathbf{X}_i^{\top}$. Measure the gap with the **Frobenius norm** $\|M\|_F$, the square root of the sum of the squared entries. Deep Dive (b) proves

$$\mathbb{E}\|\hat{\Sigma}_{\text{sim}} - \Sigma\|_F^2 = \frac{\|\Sigma\|_F^2 + (\operatorname{tr}\Sigma)^2}{n}. \tag{18.4}$$

The root-mean-square relative gap is therefore

$$\frac{\left(\mathbb{E}\|\hat{\Sigma}_{\text{sim}} - \Sigma\|_F^2\right)^{1/2}}{\|\Sigma\|_F} = \sqrt{\frac{1 + (\operatorname{tr}\Sigma)^2/\|\Sigma\|_F^2}{n}},$$

a $1/\sqrt{n}$ rate again.

EXAMPLE 4

Was a gap of 0.0250 large?

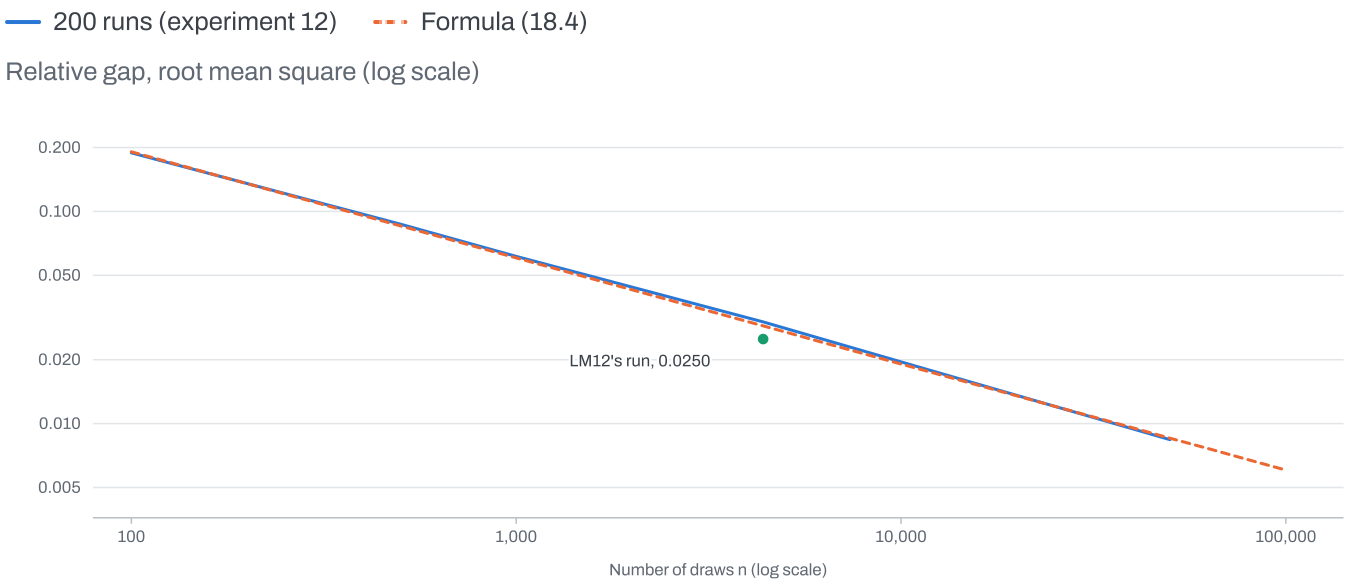
LM12's lab drew `standard_normal((4382, 5))` from `default_rng(20261011)`, coloured the rows with S5's factor L and compared `np.cov(..., ddof=0)` of the result with $\Sigma = LL^\top$. Here its 4,382 draws are n.

SOLUTION

The same draws, coloured with LM12's printed L, give the gap 0.0250. Semiconductor's simulated variance, for example, is 4.18% too high, about two standard errors of $\sqrt{2/n} = \sqrt{2/4{,}382} = 2.14\%$, since $\operatorname{Var}(Z^2) = 2$.

For S5, $(\operatorname{tr}\Sigma)^2/\|\Sigma\|_F^2 = 2.6509$, so the root-mean-square relative gap is $1.911/\sqrt{n}$: 0.0289 at $n = 4{,}382$ and 0.0085 at 50,000. In Exhibit 3, 82 of 200 runs (41%) come at or below 0.0250, so LM12's run was ordinary.

Exhibit 3: The $1/\sqrt{n}$ rate



Note: the root-mean-square relative Frobenius gap of `np.cov(..., ddof=0)` for n coloured draws, 200 runs at each n (experiment 12, `default_rng([12, 20261011])`), against (18.4); logarithmic axes, slope –1/2. At 4,382 draws the formula gives 0.0289 and the runs 0.0301 ± 0.0007, from 20 batches of 10 runs (relative standard errors below 2.7%). Of these runs, 82 of 200 ($41\% \pm 3\%$) lie at or below LM12's 0.0250. In-sample model: LM12's printed L for S5, fitted to U28's 4,382 days.

DEEP DIVE **(b) Proof of (18.4)** optional · click to expand

Let X_1 and X_2 be two entries of $\mathbf{X} \sim \mathcal{N}_N(\mathbf{0}, \Sigma)$, bivariate normal by (16.7), with standard deviations σ_1, σ_2 and correlation ρ_{12}. The matching entries of Σ are $\sigma_{11} = \sigma_1^2$, $\sigma_{22} = \sigma_2^2$ and $\sigma_{12} = \rho_{12}\sigma_1\sigma_2$. Colour them with their own 2×2 factor, for independent standard normal Z_1 and Z_2:

$$X_1 = \sigma_1 Z_1,$$
$$X_2 = \sigma_2\big(\rho_{12} Z_1 + \sqrt{1 - \rho_{12}^2}\, Z_2\big).$$

Then $X_1 X_2 = \sigma_1\sigma_2\big(\rho_{12} Z_1^2 + \sqrt{1 - \rho_{12}^2}\, Z_1 Z_2\big)$, and the two terms are uncorrelated because $\mathbb{E}[Z_1^3 Z_2] = 0$. With $\mathrm{Var}(Z_1^2) = 2$ from (14.6) and $\mathrm{Var}(Z_1 Z_2) = 1$,

$$\begin{aligned}\mathrm{Var}(X_1 X_2) &= \sigma_{11}\sigma_{22}\big(2\rho_{12}^2 + 1 - \rho_{12}^2\big) \\ &= \sigma_{11}\sigma_{22} + \sigma_{12}^2.\end{aligned}$$

The same steps give $\mathrm{Var}(X_1^2) = 2\sigma_{11}^2$, the formula with both entries equal. Each entry of $\hat{\Sigma}_{\text{sim}}$ is a mean of n iid copies of such a product, and its mean is the matching entry of Σ. By (17.1) its expected squared error is the product's variance over n.

Summing over all N^2 positions of Σ, the terms $\sigma_{11}\sigma_{22}$ add up to $(\mathrm{tr}\,\Sigma)^2$ and the terms σ_{12}^2 to $\|\Sigma\|_F^2$, which is (18.4). The call `np.cov(..., ddof=0)` centres the draws first, which changes the root-mean-square gap by under 0.4% for S5 when $n \geq 100$. ■

PITFALL **The interval measures simulation error only**

The interval of Example 3 says that more paths would bring the estimate close to 5.49%, the model's own answer. It says nothing about TAIEX, because the model assumes independent normal days fitted to the whole sample. The formula is exact under the model; every simulated number only approximates it.

導讀 **蒙地卡羅估計：誤差以 $1/\sqrt{n}$ 的速度縮小**

蒙地卡羅法（Monte Carlo method）把期望值（expectation）寫成積分（integral），再用 n 個獨立同分布（iid）抽樣的平均去估計它。強大數法則（strong law of large numbers）保證平均會收斂，標準誤（standard error）$s_Y/\sqrt{n}$ 則告訴我們誤差有多大：精確度要提高 10 倍，抽樣數就得增加 100 倍。由中央極限定理（central limit theorem），「估計值 ± 1.96 個標準誤」是 95% Monte Carlo interval，也就是只量模擬誤差的區間，不是對台股的信賴區間（confidence interval）。印出的位數要配合標準誤：0.0530 ± 0.0022 只能寫成 5.3% ± 0.2 個百分點。下一節會看到，這個速度與維度無關，所以高維度時模擬通常比格點上的數值積分（quadrature）有效率，只是單次抽樣的標準差（standard deviation）σ_Y 可能隨維度變大。

KNOWLEDGE CHECK 3

How many paths of Example 3 would bring the standard error of $\Pr(G < 1)$ down to 0.1 point?

ANSWER

The standard error is $\sqrt{p(1-p)/n}$, so $n = p(1-p)/0.001^2 = 0.0549 \times 0.9451/10^{-6}$, about 52,000 paths, five times as many as Example 3 used.

5 MONTE CARLO AGAINST QUADRATURE

- [ ] compare Monte Carlo integration with Simpson's rule, and explain why simulation wins in many dimensions

LM5 computed a normal tail probability by numerical integration, or **quadrature**. Compare it with simulation.

EXAMPLE 5

Against Simpson's rule

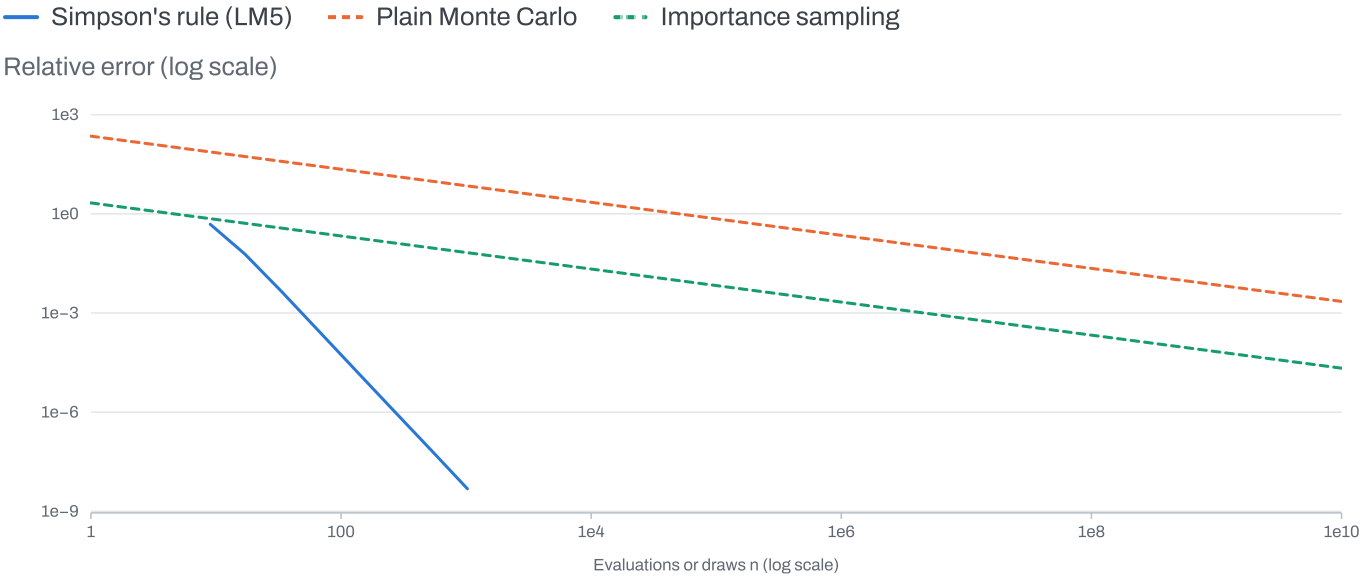
LM5's normal model gave 0050 a −5% day with probability $p = \Pr(Z \le -4.1074386) = 2.000355 \times 10^{-5}$, which LM5 approximated by Simpson's rule. Estimate p from 10^6 plain Monte Carlo draws of `default_rng([5, 20261011])`, and compare the costs.

SOLUTION

Experiment 5 draws `standard_normal(10**6)`: 18 draws fall at or below −4.1074386, against 20.0 expected, so the estimate is $1.8 \times 10^{-5} \pm 0.4 \times 10^{-5}$. Its 95% Monte Carlo interval covers p. Per draw the relative standard error is $\sqrt{(1-p)/p} = 223.6$, so with 10^6 draws it is 22%.

A 95% Monte Carlo interval of ±1% needs $(1.96/0.01)^2(1-p)/p = 1.92 \times 10^9$ draws. Simpson's rule reached a relative error of 1.23×10^{-6} with 257 evaluations of φ (LM5). Plain Monte Carlo would need about 3.3×10^{16} draws for that relative standard error. Exhibit 4 plots both rates against the evaluations.

Exhibit 4: Error against evaluations



Note: the relative error of Simpson's rule for $p = \Pr(Z \le -4.1074386)$, LM5's printed errors divided by 2.000355×10^{-5}, at 9 to 1,025 evaluations: 0.484 at 9, 1.23×10^{-6} at 257 and 4.83×10^{-9} at 1,025. The lines are the relative standard errors of plain Monte Carlo, $223.6/\sqrt{n}$, and of importance sampling (Section 6), $2.153/\sqrt{n}$. Logarithmic axes. Model: the standard normal.

Simulation wins in many dimensions. On a product grid, Simpson's rule errs by $O(\Delta x^4)$ in each coordinate of a smooth integrand, by (5.14) one coordinate at a time. With equal points on each of N axes, n evaluations give a spacing of order $n^{-1/N}$, so the error is $O(n^{-4/N})$. Monte Carlo's standard error is $\sigma_Y/\sqrt{n}$ in any dimension, though σ_Y may grow with N.

The case is an integral over $m = 1{,}231$ coordinates. A grid with two points per axis would need 2^{1231} evaluations, a number of 371 digits, while Example 3 needed 10,000 paths. Quasi-Monte Carlo methods, which use evenly spread points, are not in this book.

PITFALL **Zero hits report 0 ± 0**

With 10,000 plain draws at $k = 4.1074386$, the expected number of hits is 0.2, and no hit occurs with probability $(1-p)^{10000} = 0.8187$. In 82% of such runs the estimate and its standard error are both 0, and the interval 0 ± 0 excludes the true value. The interval (18.3) needs enough hits to estimate σ_Y, so count them first.

KNOWLEDGE CHECK 4

What is the chance that 10^5 plain draws produce no hit at or below −4.1074386?

ANSWER

The chance is $(1-p)^{100000} = 0.135$, close to e^{-2} since the expected number of hits is 2.0. About one run in seven reports 0 ± 0.

6 VARIANCE REDUCTION

- [ ] apply antithetic variates, control variates and importance sampling, and predict each one's variance reduction

Exhibit 5 shows three ways to get a smaller error from the same number of draws. Each changes what is averaged, keeps its expectation and lowers its variance.

Exhibit 5: Three ways to cut simulation error

Target, exact value	Method	Draws	Estimate	Standard error	Variance ÷
$\Pr(Z \le -4.1074386)$, 2.000355×10^{-5}	plain	10^6	1.8×10^{-5}	0.4×10^{-5}	1
	antithetic	—	—	—	1.00002
	importance sampling	10^4	2.047×10^{-5}	0.044×10^{-5}	10,786
Five-year $\mathbb{E}G$, 2.1584	plain	10,000 paths	2.165	0.010	1
	antithetic	5,000 pairs	2.155	0.004	6.06
	control $\ln G$	10,000 paths	2.161	0.003	11.4
Five-year $\Pr(G < 1)$, 0.0549	plain	10,000 paths	0.053	0.002	1
	antithetic	5,000 pairs	0.053	0.002	1.06
	control $\ln G$	10,000 paths	0.054	0.002	1.31

Note: experiments 5 and 8 (the tail), 3 (plain and control) and 6 (antithetic), seeded `default_rng([e, 20261011])` with `e` the experiment's number. The tail's antithetic row is computed, not simulated. "Variance ÷" is the plain variance per evaluation divided by the method's, from the exact formulas of this section. Model: the standard normal (the tail). In-sample model: the iid normal log returns of Example 3, fitted to all of TR23 (five years).

Antithetic variates

Since $-\mathbf{Z}$ has the same law as $\mathbf{Z}$, the partner $Y_i^- = h(-\mathbf{Z}_i)$ of $Y_i = h(\mathbf{Z}_i)$ has the same law as Y_i. Average each pair. By (16.3), $\operatorname{Var}\left(\frac{1}{2}(Y_i + Y_i^-)\right) = \frac{1}{4}\sigma_Y^2(2 + 2\rho_-)$, so for n pairs, independent across i, and $\sigma_Y^2 < \infty$,

$$\begin{aligned} &\operatorname{Var}\left(\frac{1}{n}\sum_{i=1}^{n}\tfrac{1}{2}(Y_i + Y_i^-)\right) \\ &\quad = \frac{\sigma_Y^2}{2n}(1 + \rho_-). \end{aligned} \tag{18.5}$$

Plain sampling with the same $2n$ evaluations has variance $\sigma_Y^2/(2n)$. So antithetic pairs multiply the variance by $1 + \rho_-$, and they help only when $\rho_- < 0$. When $Y = h(\mathbf{Z})$ with h non-decreasing in every coordinate (or non-increasing in every one), $\rho_- \le 0$ (stated without proof; Glasserman 2003, §4.2, p. 207).

EXAMPLE 6

Antithetic paths

Estimate the model's five-year mean growth factor $\mathbb{E}G$ with 5,000 antithetic pairs from `default_rng([6, 20261011])`, and compare with the 10,000 plain paths of Example 3.

SOLUTION

By (5.12), $\mathbb{E}G = e^{m(g+\sigma_r^2/2)} = 2.1584$. Experiment 6 draws `standard_normal((500, m))` ten times, and builds a path from each row $\mathbf{Z}$ and its partner from $-\mathbf{Z}$. The 5,000 pairs give 2.155 ± 0.004, against 2.165 ± 0.010 from the plain paths.

The partner of G is e^{2mg}/G, and G increases in every Z_t. By Problem 5, $\rho_- = -e^{-m\sigma_r^2} = -0.835$ with $m\sigma_r^2 = 0.1804$, so the variance falls by the factor $1/(1+\rho_-) = 6.06$. For $\Pr(G<1)$ the pairs give 0.0528 ± 0.0022: a path and its partner cannot both end under water, so $\rho_- = -p/(1-p) = -0.058$ with $p = 0.0549$, a factor of only 1.06.

Control variates

Suppose each draw also yields a quantity X whose mean μ_X is known exactly, such as $\ln G$ with mean mg. Then $Y - \beta(X - \mu_X)$ has the mean of Y for every constant β. By (16.3) its variance is $\sigma_Y^2 - 2\beta\sigma_{XY} + \beta^2\sigma_X^2$, smallest at $\beta = \sigma_{XY}/\sigma_X^2$. Hence, for iid pairs (X_i, Y_i) with finite variances, $\sigma_X > 0$ and known μ_X,

$$\operatorname{Var}\left(\bar{Y}_n - \beta(\bar{X}_n - \mu_X)\right) = \frac{(1-\rho_{XY}^2)\,\sigma_Y^2}{n} \qquad \text{at} \quad \beta = \frac{\sigma_{XY}}{\sigma_X^2}. \tag{18.6}$$

The best β is the least-squares slope of Y on X, so $\hat{\beta}$ comes from the draws as in (12.3) of LM12. Estimating β from the same draws adds a bias, typically of order $1/n$ (stated without proof; Glasserman 2003, §4.1.3).

EXAMPLE 7

A control variate for the five-year mean

On the paths of Example 3, use $X = \ln G$, whose mean $mg = 0.6791$ is known, as a control for $Y = G$ and for $Y = \mathbf{1}\{G < 1\}$.

SOLUTION

The least-squares slope of G on $\ln G$ is $\hat{\beta} = 2.18$ (batch standard error 0.01). Exactly, $\beta = \mathbb{E}G = 2.1584$, because completing the square as in (5.12) gives $\operatorname{Cov}(G, \ln G) = m\sigma_r^2\,\mathbb{E}G$. The controlled estimate is 2.161 ± 0.003, against 2.165 ± 0.010 plain.

The exact correlation of the control with $Y = G$ is

$$\rho_{XY} = \left(\frac{m\sigma_r^2}{e^{m\sigma_r^2} - 1}\right)^{1/2} = 0.955,$$

and with the indicator $Y = \mathbf{1}\{G < 1\}$ it is

$$\rho_{XY} = -\frac{\varphi(1.5988)}{\sqrt{p(1-p)}} = -0.488.$$

The variance falls by the factor $1/(1 - \rho_{XY}^2)$: 11.4 for G, but only 1.31 for the indicator, whose controlled estimate is 0.0535 ± 0.0020.

Importance sampling

A tail event is rare under $\mathcal{N}(0,1)$ but common under a normal shifted towards it (Glasserman 2003, §4.6). Completing the square gives $\varphi(z)e^{az - a^2/2} = \varphi(z - a)$. Substituting $x = z - a$ in (5.9) then gives, for every a and every h with $\mathbb{E}|h(Z)| < \infty$,

$$\begin{aligned} \mathbb{E}h(Z) &= \mathbb{E}\left[h(Z - a)\, e^{aZ - a^2/2}\right], \\ \Pr(Z \le -k) &= \mathbb{E}\left[\mathbf{1}\{Z \le 0\}\, e^{kZ - k^2/2}\right]. \end{aligned} \tag{18.7}$$

The second line takes $a = k$: the shifted draw $Z - k$ lands in the tail about half the time, and the weight $e^{kZ - k^2/2}$ corrects for the shift. Deep Dive (c) gives the second moment of one draw, $e^{k^2}\Phi(-2k)$ at $a = k$. LM32 applies these estimators to value at risk.

EXAMPLE 8

Importance sampling for LM5's tail

Estimate $p = \Pr(Z \le -4.1074386)$ by (18.7) with $a = k = 4.1074386$ from 10,000 draws of `default_rng([8, 20261011])`.

SOLUTION

Experiment 8 draws `standard_normal(10_000)`. Of the shifted draws $Z - k$, 5,043 land in the tail, and the weighted average is $2.047 \times 10^{-5} \pm 0.044 \times 10^{-5}$, a relative standard error of 2.1%. The exact 2.000355×10^{-5} lies inside its 95% Monte Carlo interval.

By Deep Dive (c), one draw has variance $e^{k^2}\Phi(-2k) - p^2 = 1.8546 \times 10^{-9}$, against $p(1-p) = 2.0003 \times 10^{-5}$ for a plain draw. The ratio is 10,786, so these 10,000 draws do the work of 1.08×10^8 plain ones. Three significant digits would give 10,811.

DEEP DIVE **(c) The second moment of importance sampling** optional · click to expand

One draw of (18.7) for $h(z) = \mathbf{1}\{z \le -k\}$ is $Y = \mathbf{1}\{Z \le a - k\}e^{aZ - a^2/2}$, so $Y^2 = \mathbf{1}\{Z \le a - k\}e^{2aZ - a^2}$. Completing the square, $\varphi(z)e^{2az} = e^{2a^2}\varphi(z - 2a)$, so

$$\begin{aligned}\mathbb{E}[Y^2] &= e^{a^2}\int_{-\infty}^{a-k} \varphi(z - 2a)\,dz \\ &= e^{a^2}\Phi(-k - a).\end{aligned}$$

At $a = k$ the second moment is $e^{k^2}\Phi(-2k)$, and the variance of one draw is $e^{k^2}\Phi(-2k) - p^2$. ■

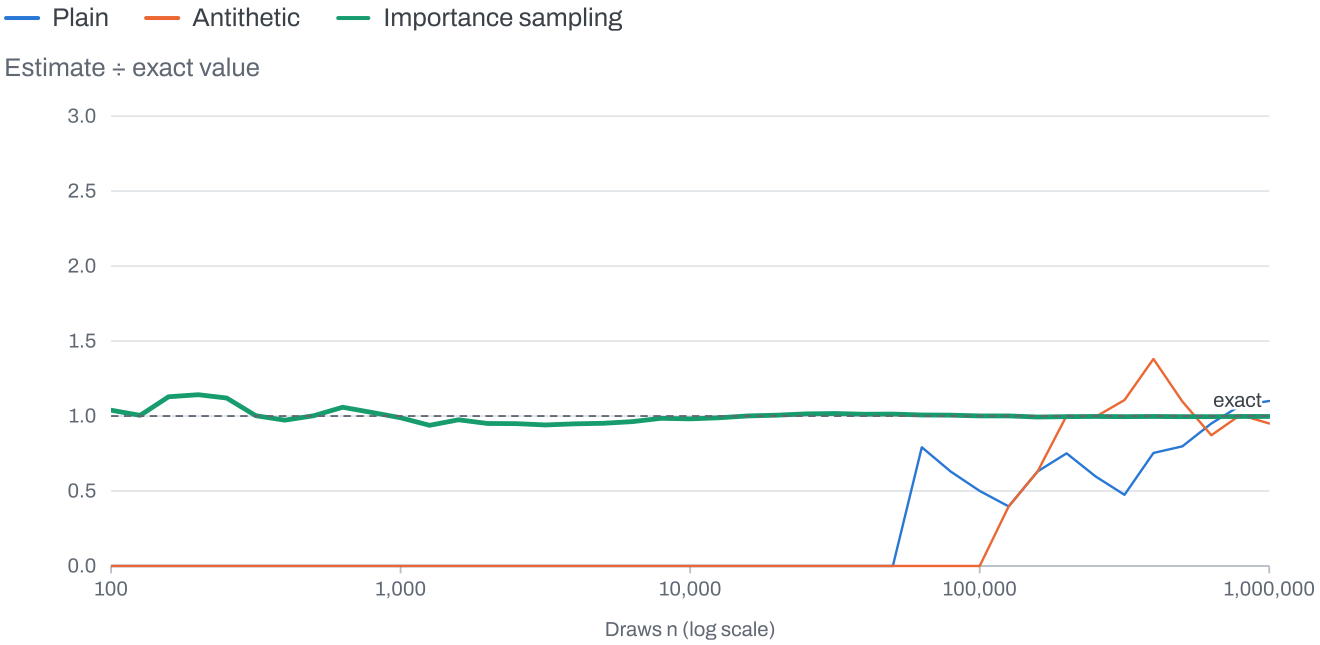
PITFALL **A shift too far out**

The relative standard error of one draw, $\sqrt{e^{a^2}\Phi(-k-a) - p^2}/p$, is 7.76 at $a = k/2$, 2.15 at $a = k$ and 132 at $a = 2k$, against 223.6 for a plain draw. Shifted too far, most draws land deep in the tail with tiny weights, and a rare draw near the threshold carries a huge one. The shifted law must also be positive wherever $h\varphi \neq 0$, as a normal always is.

The widget runs the three estimators of $\Pr(Z \le -k)$ on one stream of draws.

Three ways to estimate a tail probability

INTERACTIVE



Importance sampling estimate ± standard error	Exact Φ(−k)	Estimate ÷ exact
1.99e−5 ± 4.3e−8	**2.000e−5**	**0.997**
95% Monte Carlo interval	**Shifted draws in the tail**	**Equivalent plain draws**
covers the exact value	**499,584**	**1.08e10**

Each run draws 1,000,000 uniform numbers from the book's seeded generator (seed = run) and turns them into standard normal draws Z. All three methods use the same draws: Z for plain sampling, Z and −Z for antithetic pairs (n/2 pairs), and Z − k weighted by (18.7) for importance sampling. Values above 3 times the exact value are drawn at 3. The widget's generator is not NumPy's; the text quotes no widget value.

導讀 **變異數縮減：同樣的抽樣數，更小的誤差**

變異數縮減（variance reduction）不改變估計的期望值（expectation），只改變平均的對象。對偶變數（antithetic variates）把 $\mathbf{Z}$ 和 $-\mathbf{Z}$ 配成一對，相關係數（correlation coefficient）ρ_- 為負時才有用：估計五年成長倍數（growth factor）的期望值時，估計值的變異數（variance）縮小 6.06 倍，但尾端機率的兩個事件（event）互斥（disjoint），幾乎沒有幫助。控制變量（control variate）利用期望值已知的 $\ln G$，變異數乘上 $1-\rho_{XY}^2$，最適係數就是最小平方法（least squares）的斜率（slope）。重要抽樣（importance sampling）把抽樣用的常態分布（normal distribution）平移到尾端，再用權重（weight）修正：估計 LM5 的 −5% 尾端機率時，估計值的變異數縮小 10,786 倍。移得太遠反而更糟，靠近門檻的少數抽樣會帶著極大的權重。

KNOWLEDGE CHECK 5

Why do antithetic pairs not help with $\Pr(Z \le -k)$?

ANSWER

The pair is $\mathbf{1}\{Z \le -k\}$ and $\mathbf{1}\{Z \ge k\}$, which cannot both be 1. So $\mathbb{E}[Y_i Y_i^-] = 0$ and $\rho_- = -p^2/\big(p(1-p)\big) = -p/(1-p)$, only -2.0×10^{-5} at $k = 4.1074386$. The variance falls by the factor 1.00002: a pair costs two evaluations and is worth two plain draws, no more.

7 THE IID BOOTSTRAP

- [ ] apply the iid bootstrap to a mean and a five-year growth factor, and explain what resampling days keeps and destroys

The model of Sections 4–6 assumed normal days. The iid **bootstrap** instead takes the empirical distribution $\hat{F}$, mass $1/T$ on each of TR23's returns, as the model (Efron and Tibshirani 1993, ch. 6). A **resample** draws days with replacement, each with probability $1/T$ at every draw, and `integers(0, T, size=T)` gives the indices. By Section 3, a resample is a set of iid draws from $\hat{F}$, as in (13.13).

Under $\hat{F}$, a drawn day has mean $\frac{1}{T}\sum_t R_t = \bar{R}$ and variance $\frac{1}{T}\sum_t (R_t - \bar{R})^2 = s^2$. A resample mean $\bar{R}_{\text{boot}}$ averages T such iid days, so (17.1) gives

$$\mathbb{E}_{\hat{F}}\bar{R}_{\text{boot}} = \bar{R}, \qquad \operatorname{Var}_{\hat{F}}(\bar{R}_{\text{boot}}) = \frac{s^2}{T}. \tag{18.8}$$

Both moments in (18.8) are exact properties of the bootstrap model. LM20 derives the same $s/\sqrt{T}$ as the sampling error of $\bar{R}$.

EXAMPLE 9

The bootstrap of the mean day

With `default_rng([9, 20261011])`, draw $B = 10{,}000$ resamples of 5,849 days, and compare the spread of their means with (18.8).

SOLUTION

Experiment 9 draws `integers(0, T, size=(500, T))` twenty times. The resample means average $0.06272\% \pm 0.00016\%$, against $\bar{R} = 0.062505\%$, and their standard deviation (divisor B) is $0.01563\% \pm 0.00011\%$, with standard error $s_Y/\sqrt{2B}$. By (18.8) the exact value is $s/\sqrt{T} = 1.2084\%/\sqrt{5{,}849} = 0.01580\%$, 1.5 standard errors away. Efron and Tibshirani's (6.6) divides by $B-1$, which changes nothing here. Wasserman (2004, §8.2) divides by B in his (8.1), as this module does.

The growth factor. A resample of m days has growth factor G_{boot}, the product of $1 + R$ over its drawn days. These days are independent with mean $\bar{R}$, so (17.4) with $\mu = \bar{R}$ gives

$$\mathbb{E}_{\hat{F}} G_{\text{boot}} = (1 + \bar{R})^m. \tag{18.9}$$

The log of G_{boot} is a sum of m iid drawn log returns, with mean $m\bar{r}$ and variance ms_r^2 exactly, the two moments of the normal model of Example 3. Over 1,231 days the central limit theorem then makes the shape of one day matter little, so the two models agree.

EXAMPLE 10

The case by bootstrap

With `default_rng([10, 20261011])`, draw $B = 10{,}000$ resamples of $m = 1{,}231$ days, and estimate $\Pr_{\hat{F}}(G_{\text{boot}} < 1)$, $\mathbb{E}_{\hat{F}} G_{\text{boot}}$ and the quantiles of G_{boot}.

SOLUTION

Experiment 10 draws `integers(0, T, size=(500, m))` twenty times. Of the 10,000 resamples, 561 end under water: 0.0561 ± 0.0023, printed 5.6% ± 0.2 points.

By (18.9), $\mathbb{E}_{\hat{F}} G_{\text{boot}} = 1.00062505^{1231} = 2.1580$. The plain estimate 2.136 ± 0.009 has the 95% Monte Carlo interval 2.118 to 2.154, which misses 2.1580, as one run in twenty does. With $\ln G_{\text{boot}}$ as a control, whose mean $m\bar{r}$ is exact, the estimate is 2.158 ± 0.003.

The same draws put the average $\ln G_{\text{boot}}$ 2.4 standard errors below $m\bar{r}$. Partly for this reason, the quantiles of G_{boot} at 5%, 50% and 95% sit slightly below the normal model's 0.98 / 1.97 / 3.97. They are 0.97 / 1.94 / 3.94, with batch standard errors 0.008, 0.010 and 0.034. The standard deviation of $\ln G_{\text{boot}}$ is 0.424 ± 0.003, against $\sqrt{m}\, s_r = 0.425$ exactly.

Exhibit 6: Four answers to the case

Answer	$\Pr(G<1)$	± one standard error	5% / 50% / 95% quantiles of G	Standard deviation of $\ln G$	Assumes
Windows	4.1%	none here (LM22)	1.02 / 1.61 / 2.53	0.283	two episodes (4.75 disjoint periods)
Formula	5.49%	exact	0.98 / 1.97 / 3.97	0.425	iid normal log returns
Monte Carlo	5.3%	0.2	—	—	the same model
Bootstrap	5.6%	0.2	0.97 / 1.94 / 3.94 (± 0.008, 0.010, 0.034)	0.424 ± 0.003	iid days from $\hat{F}$

Note: the windows of Exhibit 1 are descriptive. The formula and the paths of Example 3 are an in-sample model, and so are the resamples of Example 10, which draw from all of TR23. The bootstrap quantiles carry batch standard errors. The bootstrap median sits several standard errors below 1.97 partly because this run's mean $\ln G$ lies below $m\bar{r}$; the shapes differ little. Source: FinMind `TaiwanStockTotalReturnIndex` (TAIEX), retrieved 18 October 2026.

Why the windows differ. The windows give a narrower spread of $\ln G$, 0.283 against 0.425 (Exhibit 6). They overlap, so they record two episodes in fewer than five disjoint periods, and they weight days unequally: their mean $\ln G$ is 0.502, against $m\bar{r} = 0.679$. Their error needs a bootstrap of blocks (LM22), and LM19 asks whether the spread grows like $\sqrt{m}$.

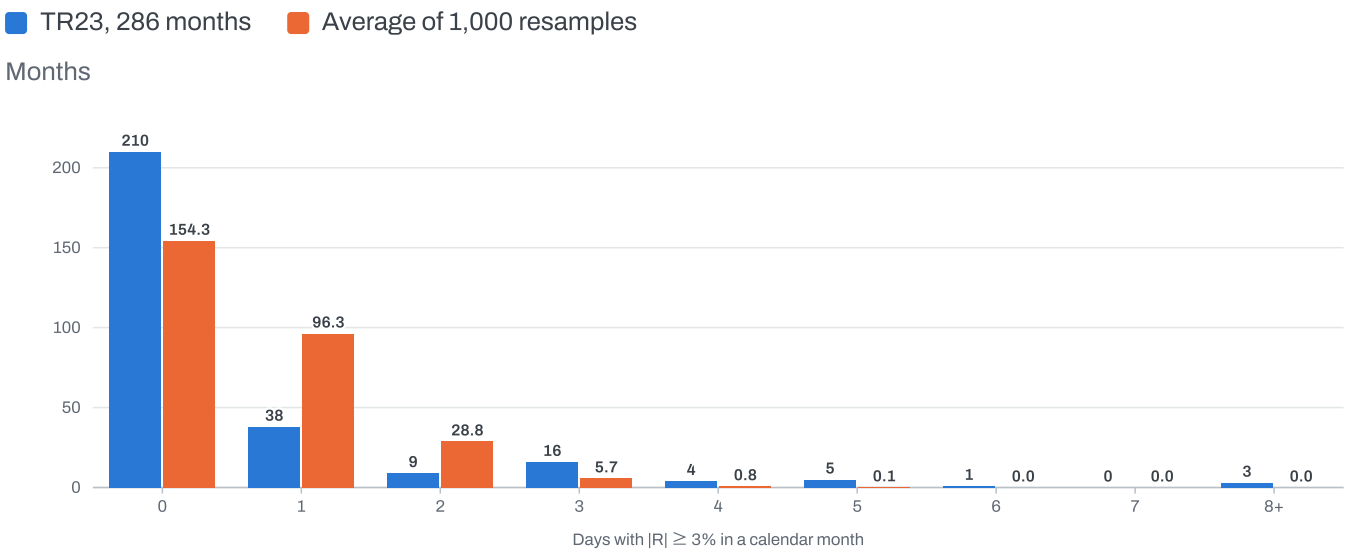
What resampling keeps and destroys

Every resampled day is one of TR23's days, so resampling keeps the one-day law exactly. The 15 days at or below −5% (LM13) recur at their own frequency, 7 April 2025 (−9.6997%) among them.

Resampling ignores the order of days, so it destroys their clustering. Exhibit 7 counts the days with $|R| \geq 3\%$ in each calendar month. TR23 has three months with eight (September 2008, October 2008 and March 2020), and a monthly variance of 1.846 (LM15). Resamples placed on TR23's dates give 0.598, the value of independent days, and the lag-1 correlation of $|r_t|$ falls from 0.161 (LM17) to about 0.

Resampling blocks of days keeps part of the clustering (LM22); volatility that depends on yesterday's is the subject of LM26.

Exhibit 7: What resampling destroys



Note: TR23's 286 calendar months by their number of days with $|R| \geq 3\%$ (LM15's counts, descriptive), the first and last partial (18 and 6 returns). Beside them, the average of 1,000 resamples of all of TR23 placed on its dates (experiment 14, `default_rng([14, 20261011])`; in-sample model), standard errors at most 0.3. The last bar counts eight or more. Monthly variance (divisor 286): 1.846 in the data, 0.598 ± 0.002 in resamples, 0.597 exactly for independent days. Lag-1 correlation of $|r_t|$ in resamples: 0.0006 ± 0.0004. Source: FinMind `TaiwanStockTotalReturnIndex` (TAIEX), retrieved 18 October 2026.

PITFALL **Resampling days is not a test on later data**

The Research hygiene page says to split by time, never at random. A bootstrap draws days at random from the whole sample, mixing both limit regimes: 3,077 returns before 1 June 2015 and 2,772 from it. It shows how a statistic varies under the iid-days model; it is no test on later data (LM34). Reading a range of resamples as an interval for TAIEX is bootstrap inference, left to LM22.

導讀 **自助重抽法：保留單日分布，破壞群聚**

自助重抽法（bootstrap）把經驗分布（empirical distribution）$\hat{F}$ 當成模型：重抽樣（resampling）是放回抽樣（sampling with replacement），每次以 $1/T$ 的機率挑一天，所以一組重抽樣就是獨立同分布（iid）的抽樣。放回重抽樣（resampling with replacement）的平均數（mean），期望值（expectation）正好是 $\bar{R}$、變異數（variance）正好是 s^2/T，成長倍數（growth factor）的期望值是 $(1+\bar{R})^m$。它保留了單日的分布，2025 年 4 月 7 日的 −9.70% 照樣可能被抽到；但它打亂了日期順序，波動的群聚（volatility clustering）完全消失：每月大波動日數的變異數從 1.846 掉到 0.598。所以自助重抽法量的是「各日獨立（independent）」這個模型下的變化，不是對未來資料的檢定。

KNOWLEDGE CHECK 6

What is the chance that 7 April 2025 is absent from a resample of 5,849 days, and what share of TR23's days appears in a resample on average?

ANSWER

Each of the 5,849 independent draws misses that day with probability $1 - 1/5{,}849$, so by (13.7) it is absent with probability $(1 - 1/5{,}849)^{5849} = 0.368$. On average about 63% of the days appear, some several times.

8 SIMULATION AND RESAMPLING IN PYTHON

- [ ] implement reproducible simulations and bootstraps with NumPy's `Generator` and SciPy, in chunks

Exhibit 8 lists the NumPy and SciPy calls of this module, with the defaults that change results.

Exhibit 8: NumPy and SciPy calls

Call	Returns	Watch
`np.random.default_rng(seed)`	a `Generator` on PCG64	a seed such as `[3, 20261011]`
`rng.random(size)`	uniform draws on $[0, 1)$	0 is possible
`rng.standard_normal(size)`	standard normal draws	
`rng.integers(low, high, size)`	integers from `low` to `high - 1`	`high` is excluded
`rng.choice(a, size)`	draws from the entries of `a`	`replace=True` by default
`rng.multivariate_normal(mean, cov)`	normal vectors	factors `cov` by SVD by default
`np.cov(x, rowvar=False)`	covariance of the columns	divisor $n - 1$ unless `ddof=0`
`np.quantile(x, u, method="inverted_cdf")`	order statistic $x_{(\lceil un \rceil)}$	the default interpolates
`stats.norm.ppf(u)`	$\Phi^{-1}(u)$	

Note: NumPy 2.5 and SciPy 1.18 references, checked against NumPy 2.5.3 and SciPy 1.18.1; `rng` is a `Generator`.

One generator per experiment, in chunks. Create the generator once, at the start of the experiment, and draw each path as a row, so that $\ln G$ is a row sum by (1.12). The code below is Example 3:

PYTHON

```python
import numpy as np
from scipy import stats

m, g, sr = 1231, 0.0005517, 0.012107          # sessions, rbar, s_r
rng = np.random.default_rng([3, 20261011])     # experiment 3, once
lnG = np.empty(10_000)
for c in range(0, 10_000, 500):                # 20 chunks of 500 paths
    z = rng.standard_normal((500, m))
    lnG[c:c + 500] = (g + sr * z).sum(axis=1)  # ln G as a sum (1.12)
y = (lnG < 0).astype(float)
print(y.mean(), y.std() / np.sqrt(y.size))     # 0.053 0.00224...
print(stats.norm.cdf(-g * np.sqrt(m) / sr))    # 0.0549...
```

Chunks keep the memory small, and they double as the batches of a batch standard error.

PITFALL **Another method draws other numbers**

A seed fixes a stream of bits, not the draws of every method. From `default_rng([2, 20261011])`, `norm.ppf(rng.random(3))` gives −1.0817, −0.0916 and −0.9382, while `standard_normal(3)` from a generator re-created with that seed gives 0.1679, 0.2891 and 0.7095. Likewise `multivariate_normal` gives other vectors than `standard_normal((n, N)) @ L.T`, because it factors the covariance matrix by SVD. Keep the method fixed when a result must reproduce.

IN PYTHON **Companion lab**

The Polars-first notebook `LM18_lab.ipynb` rebuilds the exhibits and computational examples from TR23's closes, with the seeds printed here. It solves the computational practice problems and ends by redoing the case under LM15's Student-t model.

The notebook reads FinMind files that you download once with your own token. From the repository root, run `uv run python data/fetch_finmind.py`, then `uv run python data/make_lab_data.py`. FinMind's licence does not allow the book to redistribute the data. After that, the notebook runs offline.

KNOWLEDGE CHECK 7

How much memory would 10,000 paths of 1,231 normal draws take at once, and why do chunks of 500 rows change no printed number?

ANSWER

At 8 bytes a float, $10{,}000 \times 1{,}231$ draws take 98.5 MB, and a chunk of 500 rows 4.9 MB. In the pinned NumPy, twenty calls of 500 rows consume the stream exactly as one call of 10,000 rows does, so every draw is the same. NumPy does not promise this, so `data/lm18_numbers.py` asserts it.

SUMMARY

- A seed reproduces every draw; another seed replicates the experiment.
- The inverse transform (18.1) turns uniform draws into exponential, normal and empirical draws, and colouring gives normal vectors.
- A Monte Carlo mean has standard error $s_Y/\sqrt{n}$, (18.2), and a 95% Monte Carlo interval, (18.3), for simulation error only. LM12's gap of 0.0250 was an ordinary run of (18.4).
- Quadrature wins in one dimension; in N dimensions a grid errs by $O(n^{-4/N})$, Monte Carlo by $O(n^{-1/2})$.
- Antithetic variates (18.5), control variates (18.6) and importance sampling (18.7) keep the mean and cut the variance.
- The iid bootstrap draws days from $\hat{F}$, with (18.8) and (18.9) exact. It agrees with the normal model on the case because both give $\ln G$ the same mean and variance.
- Resampling keeps the one-day law and destroys the clustering.

PRACTICE PROBLEMS

Problems on data use TR23: 5,849 daily returns, 3 January 2003 → 8 October 2026, moments with divisor T, $A = 246.12$ and $m = 1{,}231$.

1. To divide a Monte Carlo standard error by 10 with the same method, multiply the number of draws by:

   A. 10

   B. 100

   C. $\sqrt{10} \approx 3.16$

2. Plain Monte Carlo estimates $p = \Pr(Z \le -3) = 0.001350$ from $n = 10^6$ draws. The standard error of $\hat{p}$ is *closest* to:

A. 0.0367

B. 1.35×10^{-9}

C. 3.67×10^{-5}

3. For $U \sim \mathrm{Unif}(0,1)$, which is a draw from $\mathrm{Exp}(\lambda)$?

A. $-\ln(1-U)/\lambda$

B. $-\lambda \ln(1-U)$

C. $1 - e^{-\lambda U}$

4. Five returns are +1.2%, −0.8%, +0.3%, −2.5% and +0.6%. Draw from their empirical distribution with u = 0.37, 0.95 and 0.18 by the rule $R_{(\lceil 5u \rceil)}$. Why does $u = 0$ give no draw?

5. On the paths of Example 3, the plain estimates of $\mathbb{E}G$ and $\Pr(G < 1)$ have standard errors 0.0097 and 0.0022. (a) Predict them for 5,000 antithetic pairs from (18.5), with $\rho_- = -0.835$ and -0.058. (b) Under the model, prove $\rho_- = -e^{-m\sigma_r^2}$ for G and its partner e^{2mg}/G, using (5.12).

6. A control has $\rho_{XY} = 0.95$ with the simulated quantity. The number of plain draws that one controlled draw is worth is *closest* to:

A. 1.11

B. 10.3

C. 20

7. Derive the weight of (18.7) and, at $a = k$, the second moment $e^{k^2}\Phi(-2k)$ of one draw. For $k = 3$, compare the variance of one draw with that of a plain draw.

8. Prove $\mathrm{Var}_{\hat F}(\bar R_{\mathrm{boot}}) = s^2/T$ (divisor T), and evaluate its square root $s/\sqrt{T}$ with $s = 1.2084\%$. Does Example 9's 0.01563% reveal a flaw of the bootstrap?

9. Apply (17.4) under $\hat F$ to obtain (18.9), and evaluate it with $\bar R = 0.062505\%$. Compare it with $e^{m\bar r}$ for $\bar r = 0.05517\%$.

10. On a product grid in N dimensions Simpson's rule errs by $O(n^{-4/N})$ and plain Monte Carlo by $O(n^{-1/2})$. Monte Carlo has the better order when:

A. $N > 8$

B. $N > 4$

C. $N > 2$

11. Estimate the slope of $\Pr(Z \le -k)$ at $k = 2$ by (2.11), with $\Delta k = 0.01$ and 100,000 draws on each side. Given $\Phi(-2.01) = 0.0222156$ and $\Phi(-1.99) = 0.0232955$, find the exact central difference and its standard error with independent and with common draws.

12. *(Python)* With a fresh `default_rng([10, 20261011])`, draw `integers(0, T, size=(500, m))` twenty times and reproduce Example 10's $\Pr_{\hat F}(G_{\text{boot}} < 1)$ and mean G_{boot} with their standard errors. Check that one call of size $(10{,}000, m)$ gives identical indices.

SOLUTIONS

1. **B** is correct. By (18.2) the standard error is $s_Y/\sqrt{n}$, so dividing it by 10 needs $\sqrt{n}$ ten times larger, n a hundred times larger. A treats the error as $1/n$, and C inverts the square root.

2. **C** is correct. For an indicator $\sigma_Y^2 = p(1-p)$, so the standard error is $\sqrt{0.001350 \times 0.998650/10^6} = 3.67 \times 10^{-5}$. A, 0.0367, forgets n, and B, 1.35×10^{-9}, is the variance of $\hat p$, not its standard deviation.

3. **A** is correct, by (18.1) with $F_Y^{-1}(u) = -\ln(1-u)/\lambda$. B has mean λ instead of $1/\lambda$: at $\lambda = 0.02992$ its waits average 0.03 sessions instead of 33.4. C applies F_Y instead of F_Y^{-1}, and its values stay below $1 - e^{-0.02992} = 0.0295$.

4. Sorted, the returns are −2.5%, −0.8%, +0.3%, +0.6% and +1.2%. The ranks are $\lceil 1.85 \rceil = 2$, $\lceil 4.75 \rceil = 5$ and $\lceil 0.9 \rceil = 1$, so the draws are −0.8%, +1.2% and −2.5%. For $u = 0$, $\lceil 5 \times 0 \rceil = 0$ is no rank: every x has $\hat F(x) \geq 0$, so the set in $\hat F^{-1}(0)$ has no minimum.

5. (a) By (18.5), antithetic pairs multiply the variance by $1 + \rho_-$, so the standard error by $\sqrt{1+\rho_-}$: $0.0097\sqrt{1 - 0.835} = 0.0039$ for $\mathbb{E}G$ and $0.0022\sqrt{1 - 0.058} = 0.0021$ for $\Pr(G < 1)$, close to Example 6's results.

   (b) Under the model, $\ln G \sim \mathcal{N}(mg, m\sigma_r^2)$. The partner path has $\ln G^- = 2mg - \ln G$, so $GG^- = e^{2mg}$, and G^- has the law of G. By (5.12), applied to $\ln G$ and to $2\ln G$,

$$\begin{aligned} \mathbb{E}G &= \mathbb{E}G^- = e^{mg + m\sigma_r^2/2}, \\ \mathbb{E}G^2 &= e^{2mg + 2m\sigma_r^2}, \\ \operatorname{Cov}(G, G^-) &= e^{2mg} - (\mathbb{E}G)^2 \\ &= -e^{2mg}\left(e^{m\sigma_r^2} - 1\right), \\ \operatorname{Var} G &= \mathbb{E}G^2 - (\mathbb{E}G)^2 \\ &= e^{2mg + m\sigma_r^2}\left(e^{m\sigma_r^2} - 1\right). \end{aligned}$$

   The ratio of the covariance to the variance is $\rho_- = -e^{-m\sigma_r^2}$.

6. **B** is correct. By (18.6) the variance falls by the factor $1 - \rho_{XY}^2 = 1 - 0.95^2 = 0.0975$, so one controlled draw is worth $1/0.0975 = 10.3$ plain draws. A, 1.11, is $1/\rho_{XY}^2$, and C, 20, is $1/(1-\rho_{XY})$, which forgets the square.

7. Completing the square gives $\varphi(z)e^{az - a^2/2} = \varphi(z - a)$, so

$$\begin{aligned} &\mathbb{E}\left[h(Z-a)\, e^{aZ - a^2/2}\right] \\ &\quad = \int h(z-a)\, \varphi(z-a)\, dz = \mathbb{E}h(Z), \end{aligned}$$

and the weight is $e^{aZ-a^2/2}$. Deep Dive (c) gives the second moment, $e^{k^2}\Phi(-2k)$ at $a=k$. For $k=3$, $p=\Phi(-3)=1.3499\times10^{-3}$ and $e^9\Phi(-6)=7.994\times10^{-6}$, so one draw has variance 6.17×10^{-6}. A plain draw has $p(1-p)=1.348\times10^{-3}$, 218 times as much.

8. A day drawn from $\hat{F}$ has mean $\bar{R}$ and variance $\frac{1}{T}\sum_t(R_t-\bar{R})^2=s^2$. The T days of a resample are iid, so by (17.1) their mean has variance s^2/T, and $s/\sqrt{T}=1.2084\%/\sqrt{5{,}849}=0.01580\%$. No: 0.01563% lies 1.5 of its standard errors, $s_Y/\sqrt{2B}=0.00011\%$, below the exact value, an ordinary simulation error.

9. By (17.4), m independent days with mean μ give $\mathbb{E}W_m=W_0(1+\mu)^m$. Under $\hat{F}$ the resampled days are independent with mean $\bar{R}$, so $\mathbb{E}_{\hat{F}}G_{\text{boot}}=(1+\bar{R})^m$, and $1.00062505^{1231}=2.1580$. The median path is close to $e^{m\bar{r}}=e^{1231\times0.0005517}=1.9722$ by (17.6), lower than the mean path (LM17).

10. **A** is correct. Monte Carlo's order $n^{-1/2}$ beats $n^{-4/N}$ when $4/N<1/2$, that is $N>8$. B uses the trapezoid rule's order $n^{-2/N}$, and C a first-order rule.

11. By (2.11) the exact central difference is $(0.0222156-0.0232955)/0.02=-0.0540$, equal to $-\varphi(2)$ at this precision. With independent draws the variances of the two sides, 0.0222156×0.9777844 and 0.0232955×0.9767045, add up to 0.044475. With common draws the difference counts the draws in $(-2.01,-1.99]$, an indicator with probability 0.0010799 and variance $0.0010799\times0.9989201=0.0010787$. The standard errors, independent and common, are

$$\frac{\sqrt{0.044475/100{,}000}}{0.02}=0.0333,$$
$$\frac{\sqrt{0.0010787/100{,}000}}{0.02}=0.0052.$$

12. With `R` TR23's daily simple returns as a NumPy array:

```python
import numpy as np

T, m = 5849, 1231
rng = np.random.default_rng([10, 20261011])          # experiment 10
idx = np.concatenate([rng.integers(0, T, size=(500, m))
                      for _ in range(20)])
G = np.prod(1 + R[idx], axis=1)                       # G_boot per row
y = (G < 1).astype(float)
print(y.mean(), y.std() / 100)                        # 0.0561 0.0023...
print(G.mean(), G.std() / 100)                        # 2.136... 0.0094...
one = np.random.default_rng([10, 20261011])
print((idx == one.integers(0, T, size=(10_000, m))).all())   # True
```

The estimates are 0.0561 ± 0.0023 and 2.136 ± 0.009, against the exact 2.1580 of (18.9), and the comparison prints `True` in the pinned NumPy.

GLOSSARY

Term	中文	Meaning
Pseudo-random number generator	擬隨機數產生器	A deterministic recursion whose outputs behave like iid uniform draws.
Seed	隨機種子	The number, or list of numbers, that sets a generator's starting state.
Common random numbers	（兩次計算共用同一組隨機數）	The same draws in two runs whose difference is wanted, so that their errors cancel.
Uniform distribution	均勻分布	$\mathrm{Unif}(0,1)$: $\Pr(U \le u) = u$ for $0 \le u \le 1$.
Inverse transform	逆變換法	$F_Y^{-1}(U)$ has distribution function F_Y, (18.1).
Exponential distribution	指數分布	$\mathrm{Exp}(\lambda)$: $F_Y(y) = 1 - e^{-\lambda y}$, mean $1/\lambda$, median $\ln 2/\lambda$.
Monte Carlo estimator	蒙地卡羅估計量	The mean $\bar{Y}_n$ of n iid draws, an estimate of $\mathbb{E}Y$, (18.2).
Standard error	標準誤	$s_Y/\sqrt{n}$, the estimated standard deviation of $\bar{Y}_n$.
95% Monte Carlo interval	（只量模擬誤差的區間）	$\bar{Y}_n \pm 1.96\, s_Y/\sqrt{n}$, (18.3); it measures simulation error only.
Frobenius norm	Frobenius 範數	$\lVert M \rVert_F$, the square root of the sum of the squared entries.
Variance reduction	變異數縮減	A change of what is averaged that keeps the expectation and lowers the variance.
Antithetic variates	對偶變數	Pairs of draws from $\mathbf{Z}$ and $-\mathbf{Z}$, (18.5).
Control variate	控制變量	A quantity with known mean that corrects the estimate, (18.6).
Importance sampling	重要抽樣	Drawing from a shifted law and weighting the draws back, (18.7).

Term	中文	Meaning
Bootstrap	自助重抽法	Treating the empirical distribution $\hat{F}$ as the model and drawing from it.
Resampling with replacement	放回重抽樣	Drawing days from the sample, each with probability $1/T$ at every draw.

REFERENCES

- Durrett, R. (2019). *Probability: Theory and Examples*, 5th ed. Cambridge University Press, Theorem 2.4.1 (the strong law of large numbers).
- Efron, B., and R. J. Tibshirani (1993). *An Introduction to the Bootstrap*. Chapman & Hall, chapter 6, §6.2 and Algorithm 6.1 (the bootstrap estimate of standard error; their (6.6) divides by $B-1$).
- Glasserman, P. (2003). *Monte Carlo Methods in Financial Engineering*. Springer, §2.2.1 (inverse transform), §2.3.3 (multivariate normals), §4.1 (control variates; §4.1.3, the bias of an estimated coefficient), §4.2 (antithetic variates; p. 207, the monotone condition), §4.6 (importance sampling) and §9.1.2 (quantile estimation; p. 491, batches).
- Grinstead, C. M., and J. L. Snell (1997). *Introduction to Probability*, 2nd rev. ed. Providence, RI: American Mathematical Society, §5.2, Corollary 5.2 (the inverse transform for a strictly increasing distribution function).
- O'Neill, M. E. (2014). PCG: A family of simple fast space-efficient statistically good algorithms for random number generation. Technical Report HMC-CS-2014-0905, Harvey Mudd College: PCG, the paper.
- Wasserman, L. (2004). *All of Statistics: A Concise Course in Statistical Inference*. New York: Springer, §8.2 (bootstrap variance estimation; his (8.1) divides by B, as this module does).
- NumPy 2.5 reference: random `Generator`, `PCG64`, parallel streams and `SeedSequence`, compatibility policy, `Generator.integers`, `Generator.multivariate_normal`, `numpy.cov` and `numpy.quantile`, checked against NumPy 2.5.3.
- SciPy 1.18 reference: `scipy.stats.norm`, checked against SciPy 1.18.1.
- FinMind open data: `TaiwanStockTotalReturnIndex` (data_id TAIEX), retrieved 18 October 2026.